NestSports / NestAcademy — Security Re-Audit + Market Gap Analysis

Scope: `backend/`, `frontend/` (web), `NestSports/` (mobile). Method: code reading and `npm audit`. No live pen-test, no production config, no running DB. The market section comes from my own knowledge of products in this category, not from live research, so check pricing and feature claims before using them in sales material.

1. Where you stand vs. the previous audit (SECURITY_AUDIT.md, 2026-10-03)

Re-checked in code today:

Earlier finding	Status now
C-1 `/uploads` guard bypass (`/./` , `..` , `%2e`)	**Fixed.** `uploadGuard.js` resolves the path first, then judges the real folder.
C-2 NoSQL operator injection	**Mitigated.** `sanitizeInput` strips `$` keys from body/query/params globally. Still a blunt filter, see S-3.
M-1 No JWT revocation	**Fixed.** `tokenVersion` plus per-device `Session` records; password change retires old tokens.
M-4 Weak admin-set passwords	**Partly fixed.** Registration needs 8+, but the `User` schema still says `minlength: 6` .
Uploads at rest	**Good.** AES-256-GCM file encryption (`fileCrypto.js`).
Dependencies	**Still open.** `npm audit` (backend): 5 vulnerabilities (3 high, 2 moderate), including nodemailer address-parser issues. Web and mobile were not re-run.

I did not re-verify C-3, H-1 to H-5, or M-2, M-3, M-5 to M-7. Run the old checklist against the current code before relying on them.

2. Security findings that remain

S-1 · No Content-Security-Policy or security headers on the web app (High). `frontend/vercel.json` only has a rewrite. There is no CSP, `frame-ancestors` , `Referrer-Policy` or `Permissions-Policy` . Combined with S-2, one XSS gives full account takeover.

S-2 · Web session token in `localStorage` (High). `services/api.ts` stores `hrms_token` in `localStorage` . Any XSS or malicious browser extension can read it. Fix: an httpOnly, Secure, SameSite cookie, or at least a short-lived access token plus a rotating refresh token. The mobile app does this correctly (Keychain/Keystore).

S-3 · NoSQL injection defence is a global strip, not validation (Medium). It silently deletes legitimate `$` -prefixed keys. It also does not stop `{ "$" }` -free attacks such as array or object values where a string is expected (`?SN[a]=x`). Fix: validate types per route (you already have schema objects in several controllers), and cast with `String()` where an identifier is expected.

S-4 · Rate limiting is thin (Medium). - Auth is capped at 50 requests per 15 minutes per IP. That is generous for OTP and login, and there is no per-account throttle on login or OTP sending (SMS and WhatsApp cost abuse). - `/iclock` , `/api/crm` , `/internal/stats` and the WhatsApp webhooks are mounted before the limiter. - `/api/company` (register/login/upgrade) sits only under the global 5000-per-15-minute limit.

S-5 · Sensitive PII stored in plaintext in the DB (Medium). Employee `bankAccount` is a plain string. Aadhaar and PAN numbers, student `medicalNotes` , `faceDescriptor` and `deviceFaceTemplate` are also unencrypted. Files are encrypted but fields are not. A DB dump or an over-permissive read exposes it all. Fix: field-level encryption (for example Mongoose plugin with a KMS-held key) for bank, ID numbers and medical data, and never return full values to the client; return the last 4 digits only.

S-6 · Audit trail covers very little (Medium). `logAudit` is called about 14 times in total, in employee, exit, leave, biometric, security and subscription controllers. Missing: student create/edit/delete, student attendance edits, payment verify/reject, payroll run/edit, fee changes, role and permission changes, settings changes, data exports, and logins. `AuditLog` has no TTL/retention policy and no tamper-evidence (hash chain or append-only store).

S-7 · Dependency vulnerabilities (Medium). Backend: 5 open (nodemailer plus others); `xlsx` has no upstream fix (mobile and web Excel import). Move to `exceljs` or a sandboxed parser.

S-8 · Mobile hardening gaps (Medium). No certificate pinning, no jailbreak/root detection, no screen-capture protection on sensitive screens (payroll, documents, children's data; `FLAG_SECURE` on Android, a blur overlay on iOS), no app-lock (PIN/biometric re-auth after backgrounding). Keychain token storage and `allowBackup=false` are good.

S-9 · No test coverage for authorization (Medium). `tests/` has `routeGuards`, `hardening` and `units`. There are no tenant-isolation tests (user from company A requests company B's id for every resource). This is the most valuable test to add for a multi-tenant product.

S-10 · Password policy inconsistent (Low). Schema `minlength: 6`, API 8, no breached-password check, no complexity beyond length.

S-11 · Operational (Low, verify). No evidence in the repo of: DB backups and restore drills, secrets rotation, Sentry PII scrubbing, structured access logs, or alerting on failed-login spikes.

Things done well, keep them: helmet and HSTS, strict prod CORS, bcrypt, hashed reset tokens, hashed OTPs with attempt caps, 2FA with lockout plus backup codes, **passkeys (WebAuthn)**, per-device sessions, payment-event idempotency (unique gateway+paymentId index), fee-lock logic, encrypted uploads.

3. Compliance exposure (India, since you store children's data)

The Digital Personal Data Protection Act, 2023 makes a child's data (under 18) a special category: **verifiable parental consent is required, and tracking or behavioural monitoring of children is restricted**. You store minors' photos, face templates, medical notes and attendance. Gaps:

- No consent records (who consented, when, to what, which version). The only `consent_form` reference is an event document type.
- No data-subject tools: no export of "my/my child's data", no erase/anonymise endpoint, no account deletion (needed for the app stores as well; Apple requires in-app account deletion).
- No retention policy: exited students and employees keep face data and documents indefinitely.
- Face and biometric data needs its own explicit consent and a deletion path. You have `resetMyFace`, but nothing for guardians to revoke a child's face data.
- No breach-notification procedure or data-processor register (Razorpay, Meta WhatsApp, Sentry, SMTP, face service).
- No grievance officer contact in the app. A Privacy Policy exists (`/api/legal`); check it matches the real data flows.

4. What market software has that you lack

Compared against the usual product set in sports and fitness academy management: Teamup, Glofox, Mindbody, Gymdesk, PlayHQ, SportEasy, TeamSnap, LeagueApps, and Indian tools in the academy/coaching/ERP space. You already cover a lot more than most (HR, payroll, biometrics, face attendance, fee lock, WhatsApp, fixtures, inventory, events). The gaps:

Parent and student experience - In-app chat or messaging between coach and parent (you have announcements only; no two-way messaging). - Progress tracking: skill assessments, grading, belts or levels, coach feedback, goals, report cards. `sportsProfile` is a static intake form, not a time series. - Photo/video gallery per batch with parent-only visibility (you have event galleries only). - Parent self-service: pause/freeze membership, change batch, request make-up class, apply for leave for the child, view receipts and GST invoices. - Trial class / demo booking and a lead funnel.

Operations - Make-up classes, class capacity and waitlists, class packs and credits (n sessions, expiry), freeze and carry-forward. You have plans and bookings, but no credit ledger. - Coach scheduling and conflict detection, substitute coach flow, coach-to-batch workload. - Batch/timetable entity. `Student.batch` is a free-text string, so there is no capacity, schedule, venue or coach per batch. - Discounts, sibling discounts, scholarships, refunds and credit notes, security deposit, late fee, instalment/EMI plans, auto-debit (Razorpay subscriptions/UPI AutoPay). You have `OfferCode`, but check it covers sibling and scholarship rules. - GST-compliant invoicing (`Invoice` exists, but there is no GSTIN/HSN/place-of-supply fields visible, no credit notes) and Tally/Zoho export. - Lead/CRM and admissions pipeline: enquiry, follow-up, trial, conversion, source tracking. - Multi-branch / multi-centre under one owner with consolidated reports (confirm: `Company` is the tenant, so an owner with several centres may need separate accounts). - Equipment issue and return to students (inventory exists; no per-student assignment) and uniform and kit sales / POS. - Tournaments and leagues beyond fixtures: brackets, results, rankings, certificates. - Certificates and ID cards (printable, QR-verified).

Safety (strong differentiator for a children's product) - Authorised pickup list and pickup verification (named adults, photo, OTP at pickup). You are building check-out, so this is the natural next step. - Incident and injury log, first-aid record, insurance and medical clearance expiry. - Background-check and police-verification tracking for coaches, and safeguarding policy acknowledgement. - Parental waiver / e-signed consent forms with versions and timestamps. - Emergency broadcast (one tap to all parents of a batch). - Geofenced or QR-secured attendance, with an alert when a child checks in but is not collected.

Platform and admin - Granular RBAC with custom roles and per-permission toggles, plus data-scope rules (coach sees only their batches). You have fixed roles plus a Permissions screen; verify per-resource enforcement. - Offline mode for attendance on mobile with sync (grounds often have poor signal). - Reports: scheduled email/WhatsApp reports, custom report builder, retention and churn analytics, revenue forecast. - Public API / webhooks, and import/export with validation reports. - Multi-language (Hindi and regional languages) for parent screens. - Admin impersonation with audit trail, status page and in-app changelog.

5. Database model gaps

Existing models are listed in `backend/models` (59). Missing or too thin:

Proposed model	Why
`Batch` (name, sport, coach, venue/facility, weekdays, start/end, capacity, ageRange, status)	`Student.batch` and `StudentAttendance.batch` are free strings; no capacity, no schedule, no per-batch coach.
`StudentBatchEnrollment` (student, batch, from, to)	A student can move or hold several batches; keeps history.
`Guardian` / `AuthorizedPickup` (student, name, relation, phone, photo, idDoc, active, otpRequired)	`guardians` are embedded in `Student`, with no pickup authorisation or verification.
`PickupLog` (student, attendance, pickedBy, verifiedBy, method, otpVerified, timestamp)	Backs the check-out feature with proof.
`ConsentRecord` (subject, guardian, type [photo, face/biometric, medical, communications, waiver], version, grantedAt, revokedAt, ip, signature)	DPDP and waiver compliance.
`PolicyVersion` (type, version, text, effectiveFrom)	What users agreed to.
`DataRequest` (subject, type [export, erase, correct], status, dueBy, handledBy)	DPDP rights and app-store deletion.
`Incident` (student/employee, date, type, severity, description, actionTaken, reportedBy, parentNotifiedAt)	Injury and safeguarding log.
`Assessment` / `SkillProgress` (student, sport, skill, level, score, date, coach, notes) and `Certificate`	Progress tracking and report cards.
`Lead` (name, phone, source, sport, status, assignedTo, followUps[], convertedStudent) and `TrialClass`	Admissions funnel.
`Waitlist` (student/lead, batch, position, createdAt)	Capacity management.
`CreditLedger` (student, delta, reason, expiresAt, ref)	Class packs, make-up credits, refunds as credit.
`Discount` / `Scholarship` (student, type, value, from, to, approvedBy)	Beyond promo codes.
`CreditNote` / `Refund` (invoice, amount, reason, gatewayRefundId, status)	Refund and GST trail.
`Message` / `Conversation` (participants, student, body, attachments, readBy)	Two-way coach and parent chat.
`MediaItem` (batch, student[], url, visibility, uploadedBy)	Photo and video gallery with permissions.
`StaffCredential` (employee, type [first-aid, coaching licence, police check], number, expiresAt, docRef)	Compliance and expiry alerts.
`Role` / `Permission` (company, name, permissions[])	Custom roles; currently a fixed role enum on `User`.
`ApiKey` and `WebhookSubscription`	Integrations.
`LoginEvent` (user, ip, device, result, at)	Anomaly detection; `Session` only tracks live sessions.
`Branch` (company, name, address)	Multi-centre.

Fields to add to existing models: - `Student`: `schoolName`, `grade`, `allergies[]`, `medications[]`, `doctorContact`, `insuranceId`, `photoConsent`, `source` (how they found you), `guardianIds[]`. `medicalNotes` is a single text field today. - `Employee`: encrypted `bankAccount` / `aadhaarNumber` / `panNumber` plus masked display field, `policeVerificationAt`, `coachingLicence`. - `AuditLog`: `before` / `after` diff, `ip`, `userAgent`, `hash` / `prevHash`, TTL or archival policy. - `StudentAttendance`: `pickedUpBy`, `checkOutVerifyMode`; separate `checkOutMarkedBy`. At the moment one `markedBy` and one `verifyMode` cover both events. - `Invoice`: `gstin`, `hsn`, `placeOfSupply`, `taxBreakup`, `creditNoteRef`. - Indexes: `{company, status}` on `Student` / `Employee`, `{company, date}` on `StudentAttendance` (it only has a unique `{student, date}`), and a TTL on `Session` and OTP fields.

6. Recommended order of work

1. **Now (days):** `npm audit fix` on backend and re-run on web and mobile; add CSP and security headers on the web host; per-account throttle on login/OTP; align `User.password` minlength with 8; audit-log the missing high-value actions (payments, payroll, student edits, role changes, exports); tenant-isolation tests.
2. **Next (weeks):** move the web token to an httpOnly cookie or refresh-token scheme; field-level encryption for bank/ID/medical; `ConsentRecord`, account deletion and data export (also needed for Play/App Store); mobile screen-capture protection and app lock; replace `xlsx`.
3. **Product (this quarter):** `Batch` model, `AuthorizedPickup` and `PickupLog` (completes the check-out flow safely), incident log, coach-parent messaging, skill progress and report cards, credit ledger with make-up classes, lead funnel and trial classes.
4. **Later:** GST credit notes and accounting export, multi-branch, offline attendance, scheduled and custom reports, public API and webhooks, regional languages.